

AI 에이전트 구축 핵심 패턴

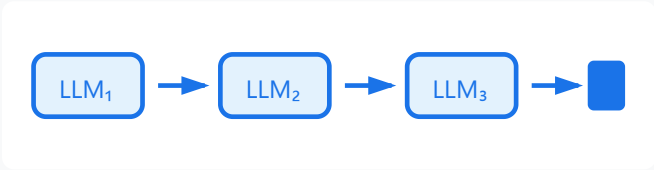


AI 에이전트를 효과적으로 구축하는 6가지 핵심 패턴

1 Prompt Chaining

프롬프트 체이닝

LLM 호출을 **순차적으로 연결**하는 가장 기본적인 패턴. 복잡한 작업을 단계별로 분해하여 처리합니다.



2 Routing

라우팅

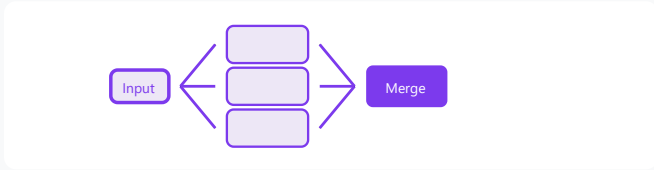
입력을 **분류**하고 적절한 **전문 에이전트**로 연결하는 패턴. 도메인별 최적화가 가능합니다.



3 Parallelization

병렬화

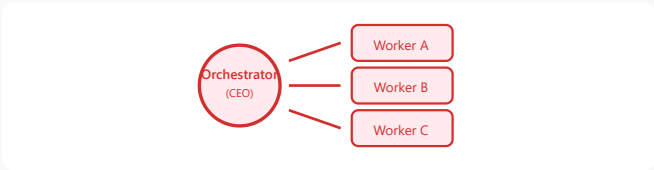
작업을 **동시에 처리**하여 속도(Sectioning)와 신뢰도(Voting)를 높이는 패턴입니다.



4 Orchestrator-Workers

오케스트레이터-워커

중앙 조율자가 **동적으로 작업을 분배**하는 패턴. 사전에 작업 목록을 모를 때 효과적입니다.



5 Evaluator-Optimizer

평가자-최적화자

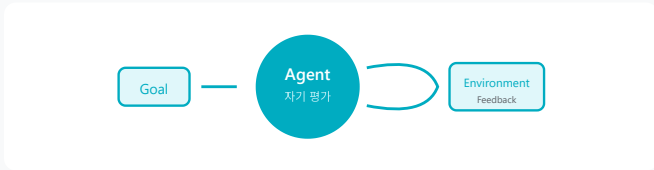
생성과 평가를 **반복**하며 품질을 개선하는 패턴. 명확한 기준이 있을 때 효과적입니다.



6 Autonomous Agent

자율 에이전트

최소한의 입력으로 **독립적으로 작동**하는 가장 진보된 패턴. 환경 피드백 기반으로 스스로 판단합니다.



"가장 단순한 것부터 시작하라" - 필요할 때만 복잡성을 추가하세요. 이 패턴들은 독립적으로 사용하거나 조합할 수 있습니다.

Workflow vs Agent

Workflow

사전 정의된 시스템

사전에 정의된 순차적 단계를 따르는 자동화 시스템.
마치 요리 레시피처럼 항상 동일한 순서로 실행됩니다.

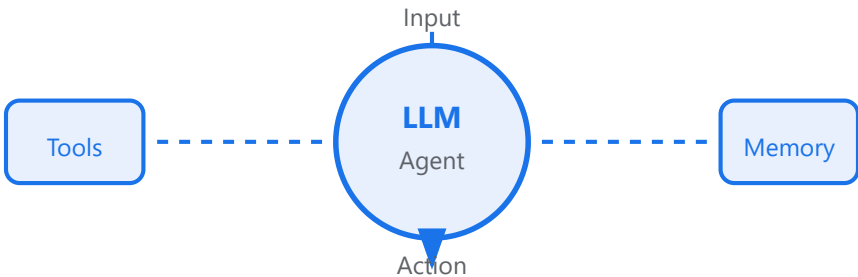


고정된 순서 예측 가능 빠른 실행 저비용

Agent

자율적 의사결정 시스템

환경을 인식하고, 정보를 처리하며, **자율적으로 행동**하여
목표를 달성하는 시스템. 상황에 따라 유연하게 대응합니다.



동적 의사결정 컨텍스트 인식 도구 활용 자기 평가

AI 에이전트는 단일 개체가 아닌 **여러 서브 에이전트의 집합**으로 구성되며, 멀티 에이전트 시스템 내에서 협력하여 하나의 통합된 에이전트처럼 동작합니다.

When to Use Agents

🕒 Agent가 적합한 경우

동적 대화 Dynamic Conversations

맥락에 따른 판단이나 개인화된 추천이 필요한 상호작용

예시: 맞춤형 제품 추천, 복잡한 상담

고가치 의사결정 High-value Decisions

잘못된 결정의 비용이 에이전트 운영 비용보다 훨씬 큰 경우

예시: 인프라 설계 최적화, 금융 심사

복잡한 워크플로우 Unpredictable Workflows

"if-then" 규칙으로 모든 경우를 커버하기 어려운 작업

예시: 진단 시스템, 동적 변수 처리

📋 Workflow가 적합한 경우

고빈도 단순 작업 High-frequency Tasks

추론보다 속도와 규모가 더 중요한 반복 작업

예시: DB 조회, 정형 메시지 파싱, FAQ 응답

예측 가능한 프로세스 Predictable Processes

입력과 출력이 명확하고 순서가 고정된 작업

예시: 환불 처리, 비밀번호 재설정

비용/지연 민감 Cost & Latency Sensitive

에이전트의 추가 비용과 지연을 감당하기 어려운 경우

예시: 대량 데이터 처리, 실시간 응답



핵심 원칙: 에이전트가 이론적으로 적용 가능하더라도, 많은 경우 전통적인 워크플로우가 더 나은 성능을 보입니다. 복잡성이 명확한 가치를 제공할 때만 에이전트를 도입하세요.

Agent Components



Models

LLM / AI 모델

추론, 의사결정, 다양한 모달리티 처리가 가능한 핵심 지능. 복잡도와 비용에 따라 적절한 모델 선택이 중요합니다.

- GPT-4/5
- Claude
- Gemini



Tools

도구 / API 연동

모델의 기능을 확장하는 외부 시스템 연동. 웹 검색, 파일 처리, 외부 API 호출 등을 수행합니다.

- Web Search
- MCP
- APIs



Knowledge & Memory

지식 베이스 / 메모리

정적 지식(문서, 정책)과 영속적 메모리(대화 기록)를 저장하고 검색하여 컨텍스트를 유지합니다.

- Vector DB
- Embeddings
- RAG



Guardrails

안전장치 / 제약조건

에이전트가 의도한 대로 작동하도록 보장하는 안전장치. 부적절하거나 관련 없는 응답을 방지하고 일관성을 유지합니다.

- Content Filters
- LangChain Guardrails
- Guardrails AI



Orchestration

조율 / 관리

여러 서브 에이전트가 함께 작동하는 방식을 관리하고, 배포 및 모니터링을 담당합니다. 지속적인 업데이트와 개선이 핵심입니다.

- LangChain
- CrewAI
- LlamaIndex

주요 AI 기업들이 제시하는 구성요소를 기반으로 하되, **LangChain**, **CrewAI**, **LlamaIndex** 등 다양한 프레임워크로 구현 가능합니다.

Prompt Chaining

핵심 개념

작업을 **일련의 단계**로 분해하여 각 LLM 호출이 이전 호출의 출력을 처리하는 방식입니다. 마치 **조립 라인**처럼 순차적으로 작업을 수행하며, 필요 시 조건 분기를 통해 흐름을 제어할 수 있습니다.

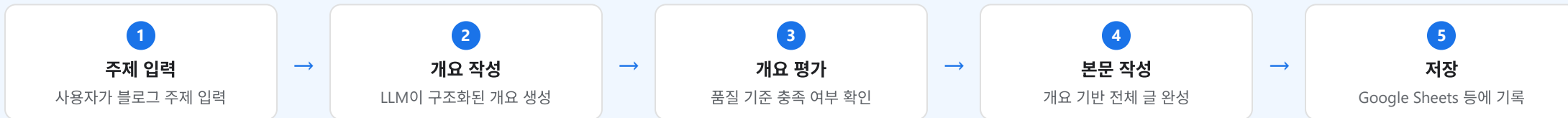
언제 사용하나요?

- 작업이 자연스럽게 순차적 하위 작업으로 분해될 때
- 각 단계가 최종 결과에 의미있게 기여할 때
- 단계별 추론이 정확도를 높일 때
- 프로세스 전체에 품질 체크포인트가 필요할 때

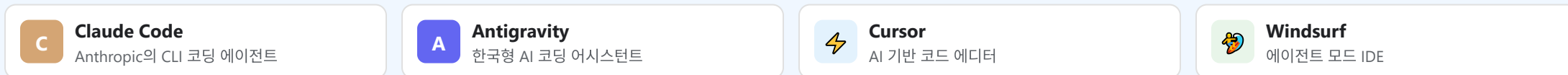
워크플로우 구조



📄 실제 예시: 블로그 글 자동 생성



<> 실제 예시: AI 코딩 에이전트

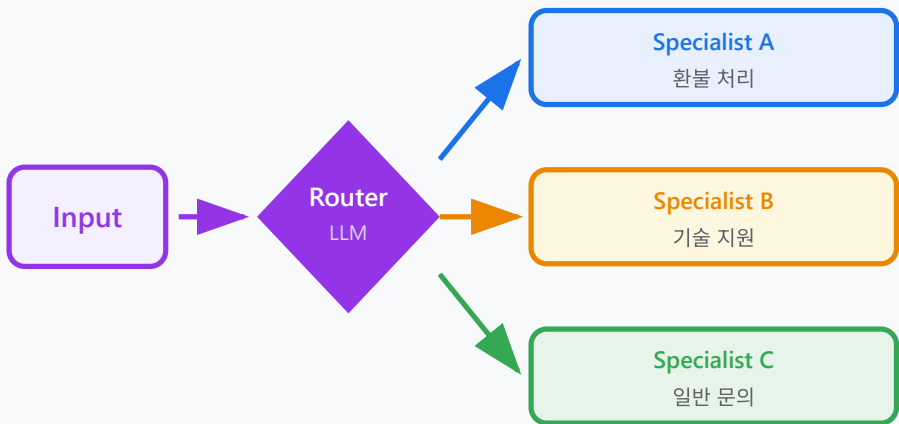


핵심: 각 LLM 호출이 이전 단계의 출력을 입력으로 받아 순차적으로 처리하며, 조건 분기(Gate)를 통해 품질 제어가 가능합니다.

핵심 개념

입력을 분류(Classification)하고, 해당 유형에 특화된 전문 에이전트나 프롬프트로 라우팅하는 패턴입니다. 각 전문가가 자신의 영역에서 최적의 결과를 제공합니다.

라우팅 워크플로우 구조



실제 예시: 고객 서비스 봇



Router (GPT-4o mini)

사용자 입력을 분류하여 적절한 에이전트로 전달



"환불 요청" → Refund Agent

환불 정책 확인 및 처리 프로세스 안내



"오류 발생" → Tech Support Agent

기술적 문제 진단 및 해결 방법 제시



"영업 시간?" → General FAQ Agent

일반 문의사항에 대한 빠른 응답

적합한 사용 사례

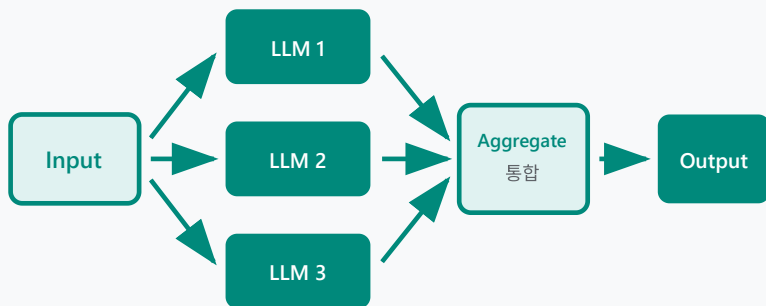
- ✓ 다양한 입력 유형 - 전문화된 처리가 필요한 경우
- ✓ 리소스 최적화 - 간단한 쿼리는 저비용 모델로
- ✓ 도메인 전문화 - 영역별 전문 지식 적용



Sectioning

작업 분할 방식

하나의 작업을 독립적인 하위 작업으로 분할하여 동시에 실행합니다. 각 결과는 최종적으로 통합됩니다.



예시: LinkedIn URL 수집

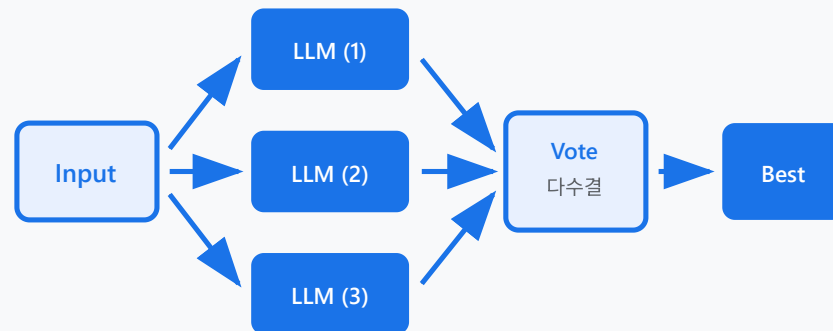
여러 웹사이트의 LinkedIn URL을 동시에 검색하여 Google Sheet에 저장. 순차 실행 대비 **3-5배 빠른 처리** 가능.



Voting

다수결 / 앙상블 방식

동일한 작업을 여러 번 실행하고 결과를 집계합니다. 다양한 출력을 얻거나 다수결로 신뢰도를 높입니다.



예시: 코드 리뷰

동일 코드를 3개 LLM이 검토 후 다수결로 취약점 판단. 단일 실행 대비 **오탐률 70% 감소**.

순차 실행

30초

병렬 실행

10초

속도 향상

3x 빠름

신뢰도 (Voting)

+40%

핵심: Sectioning은 속도와 효율성을, Voting은 정확도와 신뢰성을 높입니다. n8n에서는 "Run once for each item" 설정으로 쉽게 구현 가능합니다.

Orchestrator-Workers

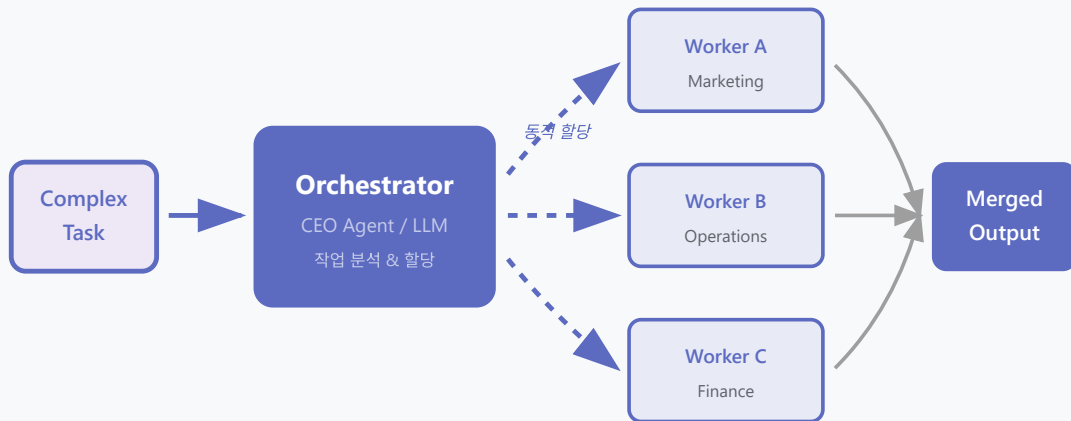
핵심 개념

중앙 **오케스트레이터**가 작업을 분석하고, 필요한 하위 작업을 **동적으로 결정**하여 전문 워커들에게 할당합니다.

병렬화와의 차이

병렬화는 **사전 고정된 작업 목록**을 처리하지만, 오케스트레이터-워커는 **실행 중 작업을 결정**합니다.

동적 작업 할당 구조



실제 예시: 협업 영업 계획



병렬화(Parallelization)와의 핵심 차이

병렬화: "3개 파일을 동시에 처리해" (고정된 작업)

오케스트레이터: "이 문제를 분석해서 필요한 작업을 찾고, 적절한 전문가에게 맡겨" (동적 결정)

핵심: 오케스트레이터는 작업을 **동적으로 분해**하고, 각 워커는 자체 메모리와 도구를 가진 독립적인 에이전트입니다. 실행 중 결과를 통합하여 최종 출력을 생성합니다.

핵심 개념

Generator(생성자)가 결과물을 만들고, **Evaluator(평가자)**가 품질을 검토합니다.
기준 미달 시 피드백과 함께 재생성을 요청하는 **반복 개선 루프**를 형성합니다.

언제 사용하나요?

- 명확한 품질 기준이 존재할 때
- 반복적 개선이 가치를 더할 때
- 자동화된 품질 보증이 필요할 때
- 사람의 검토 없이 일정 수준 보장이 필요할 때

평가-최적화 피드백 루프



핵심: Evaluator가 명확한 기준으로 품질을 판단하고, 미달 시 구체적인 피드백과 함께 Generator에게 재작업을 요청합니다. 무한 루프 방지를 위해 최대 반복 횟수를 설정하세요.

Autonomous Agent

핵심 개념

최소한의 입력(보통 단일 목표)만으로 독립적으로 작동합니다. 에이전트는 스스로 행동을 취하고, 환경 피드백을 관찰하며, 목표 달성 여부를 자체 평가합니다.

자율 에이전트 실행 루프



<> 실제 예시: AI 코딩 에이전트

**Cursor**

AI 기반 코드 에디터

**Windsurf**

에이전트 모드 IDE

자율 에이전트의 핵심 특징

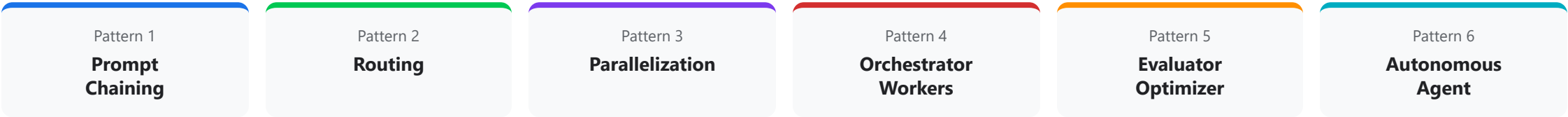
- ✓ 파일 구조 자동 생성 - 프로젝트 구조 설계
- ✓ 멀티파일 편집 - 여러 파일 동시 수정
- ✓ 터미널 명령 실행 - 빌드, 배포 자동화
- ✓ 환경 적응 - 오류 발생 시 자동 수정

실제 사례: Windsurf API 배포

Heroku에 API를 배포하는 과정에서 Windsurf가 [PostgreSQL Hobby Dev](#) 애드온 지원 중단을 감지하고, 자동으로 [PostgreSQL Essential 0](#) 으로 재설정하여 배포를 완료했습니다.

핵심: 자율 에이전트는 환경 피드백을 기반으로 스스로 판단합니다. 개념적으로 단순하지만, 실제로는 매우 복잡하고 다양한 행동을 생성할 수 있습니다.

Summary & Key Principles





Keep Architecture Simple

가장 단순한 해결책부터 시작하세요. 에이전트 시스템의 복잡성을 점진적으로 추가하는 것이 핵심입니다.

- 단일 LLM으로 시작
- 필요할 때만 패턴 조합
- 과도한 엔지니어링 지양



Make Reasoning Visible

에이전트의 사고 과정을 투명하게 노출하세요. 디버깅과 개선에 필수적입니다.

- 중간 결과물 로깅
- 의사결정 근거 기록
- 실패 지점 추적 가능



Ensure Reliable Tool Interactions

도구 호출의 신뢰성을 확보하세요. 에이전트 성공의 핵심 요소입니다.

- 명확한 도구 스키마 정의
- 오류 처리 메커니즘
- 도구 호출 결과 검증

6가지 패턴을 조합하여 복잡한 문제를 해결하세요

각 패턴은 독립적으로 사용하거나, 필요에 따라 조합할 수 있습니다.


Input

→


Patterns

→


Solution

Building Effective Agents | 6 Composable Patterns for AI Agent Architecture

실습파일 안내



JUPYTER NOTEBOOK

ai_agent_patterns_examples.ipynb

6가지 AI 에이전트 핵심 패턴을 직접 실행하고 검증할 수 있는 실습 노트북입니다.

Pattern 1 Prompt Chaining

Pattern 2 Routing

Pattern 3 Parallelization

Pattern 4 Orchestrator-Workers

Pattern 5 Evaluator-Optimizer

Pattern 6 Autonomous Agent